

Revenue Streams — business-funded, never a customer markup

Prepared 2026-07-30. The design goal: **customers pay wholesale + a discount, never a markup.** Every dollar the platform earns comes from **businesses** (advertisers, sellers, sponsors, brands, developers, financing partners) or from **structural margin** (breakage, negotiated spreads). All of it is written to one ledger — **RevenueEvent** via `recordRevenue()` (`backend/sdk/revenue.ts`) — which carries the invariant `customer_paid: false` . The admin `revenueReport` proves that invariant (expects `customer_paid_usd = 0`).

The advertiser AdGrid alone (~\$3,540 margin per matched user/year) already out-earns the retired markup (~\$146/user/year) by ~24×, so dropping the customer markup is affordable many times over — the constraint is advertiser supply, not money.

PPC AdGrid is open to advertisers across all tiers (2026-09-11). Every advertiser — Tier 1 (Founding), Tier 2 (Scale) and Tier 3 (Unlimited) — can run PPC placements on the premium AdGrid: users watch the advertiser's product, answer its questions, and are credited, and the advertiser pays per engagement. This widens advertiser supply into the AdGrid across the whole tier stack.

What's wired

Businesses interact through the new **BusinessPortal** page (sign up, subscribe, buy placement, book a panel). Functions: `businessSignup` , `businessSubscribe` , `buySponsoredPlacement` , `createAudiencePanel` , `revenueReport` . Entities: `RevenueEvent` , `BusinessAccount` , `BusinessSubscription` , `SponsoredPlacement` . All fees live in the **Revenue** settings category (safe 0/off defaults — nothing bills until you price it).

The streams (owner-selected set)

Option A — recover the margin from businesses

#	Stream	How it's implemented	Owner action
A1	AdGrid funds all discounts	<code>ADGRID_FUNDS_ALL_DISCOUNTS</code> flag + <code>adgridFundsAllDiscounts()</code> — routes the advertiser/loyalty pool to fund the discount for every user (extends the loyalty engine you already have).	Turn on; sign advertisers to grow the pool.
A2	Seller-side commission	<code>splitSellerProceeds()</code> + <code>MARKETPLACE_SELLER_COMMISSION_PCT</code> (0.10) — takes the platform cut from the seller's proceeds, never the buyer. Records <code>seller_commission</code> .	Wire <code>splitSellerProceeds</code> into the marketplace sale/settlement (one line where seller funds release).
A3	Sponsored placement	<code>buySponsoredPlacement</code> + <code>SponsoredPlacement</code> entity + <code>SPONSORED_PLACEMENT_PRICE_USD</code> . Customer prices unchanged.	Set a price; (optionally) boost sort for active placements.
A4	Below-wholesale sourcing	<code>recordRevenue({type:"sourcing_margin"})</code> hook. The spread between wholesale and retail is margin.	Negotiate wholesale; log the margin per fulfilled order.
A5	Affiliate commissions	Existing affiliate listings; log via <code>recordRevenue({type:"affiliate_commission"})</code> .	Grow affiliate catalog (already supported).
A6	Business sign-up / onboarding fee	<code>businessSignup</code> + <code>BUSINESS_SIGNUP_FEE_USD</code> / <code>BUSINESS_ONBOARDING_FEE_USD</code> .	Set the fee.
A7	B2B SaaS tiers	<code>businessSubscribe</code> + <code>BusinessSubscription</code> + <code>BUSINESS_SAAS_BASIC/PRO/ENTERPRISE_USD</code> (49/299/999).	Set tier prices; recurring billing job when the processor is live.
A10	Lead / referral fee	<code>computeLeadFee()</code> + <code>LEAD_REFERRAL_FEE_USD / _PCT</code> ; record <code>lead_fee</code> when the platform sends a business a customer.	Set the fee; call <code>recordRevenue</code> from the affiliate/referral/booking flow.
A11	Co-op marketing funds	<code>recordRevenue({type:"coop_fund"})</code> ; brand dollars used <i>as the customer discount</i> .	Sign co-op deals; apply funds to the discount pool.
A12	Payment-processing rebate	<code>processingRebatePct()</code> + <code>PROCESSING_REBATE_PCT</code> ; record	Negotiate the processor rebate; then set the %.

processing_rebate on volume.

Option B — other non-customer revenue

#	Stream	How it's implemented	Owner action
B13	Advertising	buySponsoredPlacement with a slot → records advertising.	Define ad slots + price.
B14	Breakage	revenueReport estimates it: outstanding closed-loop points × BREAKAGE_RECOGNITION_PCT (0.15). Reporting-only.	Review the estimate; recognize per your accounting.
B15	Sponsored jackpots/prize competitions	recordRevenue({type:"sponsored_prize"}); a brand funds the prize pool for exposure.	Add a sponsor to a prize pool; log the sponsorship.
B16	Developer/creator cut	devCreatorCutPct() + DEV_CREATOR_PLATFORM_CUT_PCT (0.20); record dev_creator_cut.	Wire into the dev/creator payout (apply the cut, log it).
B17	White-label / API licensing	recordRevenue({type:"white_label"}); existing APIAccessKey.	Close licensing deals; log the fee.
B18	BNPL merchant fee	Wired in affirmConfirm — records bnpl_merchant_fee = order × BNPL_MERCHANT_FEE_PCT.	Set the % your BNPL partner pays.
B19	Premium membership fee	Existing \$1/day-from-earnings (membershipDailyFee).	None — already live.
B20	Gift-card / catalog arbitrage	recordRevenue({type:"arbitrage_margin"}); buy below face, redeem at face.	Source bulk; log the spread.
B22	Shipping spread	recordRevenue({type:"shipping_margin"}); negotiated rate below what's passed through.	Negotiate carrier rates; log the spread (keep it honest).
B23	Audience panels / segments	createAudiencePanel + SponsoredPlacement(slot:"audience_panel") + AUDIENCE_PANEL_PRICE_USD.	Set a price; deliver aggregate, consented insights only.

The closed-loop margin model (seller keeps 100% + cash-back)

The default marketplace mode is now **cashback** (MARKETPLACE_MARGIN_SOURCE): the member seller keeps **100%** of their sale **and** gets **10% back in non-cashable points** (SELLER_CASHBACK_POINTS_PCT), and the **buyer pays no markup**. The platform's 10% is not taken from anyone — it comes from the closed loop:

- Seller cash-back (Suggestion 1)** — issued as closed-loop points (nearly free scrip). Recorded as a **subsidy** (recordSubsidy, kind:"subsidy"), i.e. a cost, not revenue — so it never inflates the revenue total. **The cash-back is gated behind using the site as a member:** while SELLER_CASHBACK_REQUIRES_ACTIVATION is on (default), the 10% lands in a **locked** pending_cashback_points bucket the seller can't spend until they complete the **one-click seller onboarding** — agreeing to use the platform as **both a seller and a member for a year** (SELLER_USER_COMMITMENT_MONTHS, default 12). sellerActivateMembership records that consent to the append-only ConsentRecord ledger and **sweeps** the locked points into spendable points. This keeps the perk fully inside the closed loop — a seller can only realize their cash-back by using the site — and is *not* money transmission: nothing is ever converted to cash.
- Breakage is the real margin (Suggestion 2)** — breakageReport tracks outstanding vs redeemed points and recognizes the unredeemed portion (BREAKAGE_RECOGNITION_PCT) as retained value. This is where the "10%" actually lands: value of points that are never redeemed, paid by no one.
- Catalog sourcing spread (Suggestion 3)** — when a platform-catalog item is bought with points, the spread between the points' face value and the wholesale cost (wholesale_cost_usd, or CATALOG_WHOLESALE_FRACTION) is booked as sourcing_margin revenue — funded by the cash the buyer already earned.
- Advertiser pool backstop (Suggestion 4)** — breakageReport proves breakage + the advertiser pool (pooledAnnualRevenueUsd) **cover** the cash-back subsidies (coverage() → seller_cashback_is_free), so the perk is genuinely funded, not magic.

The one door kept shut: **no owner-side points→cash conversion**. Points stay non-cashable/closed-loop — that's the money-transmission shield. The platform's real money is the survey/advertiser cash and the breakage, which are already the platform's; it never needs to launder value back through points.

Modes: set MARKETPLACE_MARGIN_SOURCE to **seller** (commission from the seller, A2) or **off** (seller keeps 100%, no cash-

back) if you prefer. `revenueReport` now splits `recorded_revenue_usd` from `subsidies_usd` and shows `net_after_subsidies_usd`.

Member storefronts — every user can sell in one click

Because fulfillment is AI-automated and the economy is closed-loop, **any user can become a seller in one tap** (`sellerSignupOneClick`) — the account username is the seller name; there's no separate seller account. The deal, advertised prominently (`SellerDealBanner` on the Marketplace and seller pages):

- **Your own uploads:** keep **100%** of the sale + **10% back in points** (the seller cash-back model above — 10% locked until member activation, funded by breakage + the advertiser pool).
- **Curated resale (`addCatalogToStorefront`):** a user resells a **platform-catalog** product they found via search from their storefront. The **platform sources + fulfills** it (AI order function) and keeps the **wholesale spread** (`sourcing_margin`); the buyer pays no added markup; the **curator earns 10% back in points** on a real sale (`curator_reward` subsidy, `CURATOR_REWARD_POINTS_PCT`, locked until activation). This is *not* the “keep 100%” deal — the curator didn't source or ship it, so they get the 10% only.

Why this is safe and non-farmable: the 10% pays **only on a real sale** (`purchaseMarketplaceListing`), never for listing activity — so bulk-adding products mints nothing. Curated listings disclose platform fulfillment (`fulfilled_by: "platform_ai"`). The optional **Active Seller** level (≥ 300 products across ≥ 30 days) is recognition only and grants **no** points. And the one door stays shut: points remain non-cashable — a seller realizes value only by spending on the site.

Compliance notes

- **Never a customer markup** is the invariant — `revenueReport.customer_paid_usd` must stay 0.
- **Data products (A5-adjacent, B23):** any insight sold to a business must be **aggregate + anonymized + consented** (your Privacy Policy + counsel). Never sell individual-level data.
- **Breakage (B14)** is a reporting estimate, not a cash booking — recognize per your accountant.
- Avoid **float/interest on held balances** — that's a money-transmission trap (not implemented here).
- **Membership + AdGrid** language must keep FTC-clean disclosure (already reviewed in the lawyer packet).

Where to grow next (documented one-liners, not yet wired into money flows)

A2 seller commission and B16 dev/creator cut ship as **helpers + settings + ledger hooks** so they don't touch the live payment/payout paths until reviewed. Wiring each is a one-line `recordRevenue()` + `splitSellerProceeds()` / `devCreatorCutPct()` call at the point where seller funds release / creator payouts run — say the word and I'll wire them into those specific flows with a verification pass.

Ad-network metrics + demographic & audience targeting (2026-09 update)

Full network-standard metric set (measured, never guaranteed). Advertisers now get the complete mobile-ad-network metric set, computed from real platform activity: **eCPM, CPM, IPM, CPP**, and the windowed **ROAS curve (D1 / D3 / D7 / D14 / D28 / D90 / D365)**, plus the publisher-side yield metrics — **fill rate, ARPDAU and retention (D1 / D7 / D28)**. Every figure is measured and shown with its basis; the platform never guarantees an ROI. (Engine: `ad-metrics.ts`; on-demand read: `adMetricsReport`.)

Tracked over time and AI-optimized. A scheduled sweep (`adMetricsSweep`) tracks each metric as history (`OptimizationSignal` + `AgentLearningMemory` — no new tables) and runs an AI optimizer that biases delivery from measured results. Every automatic action is reversible and routed through the autonomy kernel (`ad_optimization`): it auto-applies on trust or queues a human review, and it **never raises spend and never touches billing or payouts** (those stay permanently gated).

Demographic + new-vs-existing targeting. On top of the interest/behavior cohorts, advertisers can target **demographics — age range, gender, country and region** — and **audience type (new vs existing users)**, matched against users' own non-identifying profile data (never an individual). Advertisers set an `optimize_for` objective — **ROAS, new-user acquisition, or existing-user retention** — that steers the AI delivery optimizer. (Engine: `ad-audience.ts`.)